Future Generation Computer Systems 162 (2025) 107484

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

Review article

Small models, big impact: A review on the power of lightweight Federated Learning

Pian Qi, Diletta Chiaro, Francesco Piccialli *

Department of Mathematics and Applications "R. Caccioppoli", University of Naples Federico II, Naples, Italy

ARTICLE INFO

Keywords:
Federated Learning
Device heterogeneity
Constrained devices
Lightweight federated learning
Tiny federated learning

ABSTRACT

Federated Learning (FL) enhances Artificial Intelligence (AI) applications by enabling individual devices to collaboratively learn shared models without uploading local data with third parties, thereby preserving privacy. However, implementing FL in real-world scenarios presents numerous challenges, especially with IoT devices with limited memory, diverse communication conditions, and varying computational capabilities. The research community is turning to lightweight FL, the new solutions that optimize FL training, inference, and deployment to work efficiently on IoT devices. This paper reviews lightweight FL, systematically organizing and summarizing the related techniques based on its workflow. Finally, we indicate potential problems in this area and suggest future directions to provide valuable insights into the field.

Contents

* Corresponding author.
E-mail address: francesco.piccialli@unina.it (F. Piccialli).

https://doi.org/10.1016/j.future.2024.107484
Received 5 July 2024; Received in revised form 14 August 2024; Accepted 16 August 2024
Available online 23 August 2024

1. Introduction

Federated learning (FL) represents a paradigm shift in the way machine learning (ML) models are trained, emphasizing data privacy and security by allowing individual devices to collaboratively learn a shared model without centralizing raw data [1]. FL decentralizes the model training process across multiple devices, such as mobile and Internet of Things (IoT) devices. Each device independently trains the model with its local data and then uploads only the updated model parameters, rather than the raw data, to a central server for model aggregation. This method greatly reduces privacy risks because the data remains stored locally on each device, eliminating the need to transfer sensitive information to a centralized server. The decentralized nature of FL mitigates the risks related to data breaches and compliance issues, making it particularly attractive in sensitive domains such as healthcare [2,3], finance [4,5], autonomous driving [6], and mobile computing [7] (see Fig. 1).

However, implementing FL in real-world scenarios presents several challenges, with device heterogeneity being particularly prominent [8]. Devices participating in FL can vary significantly in terms of computational capabilities, memory, battery life, and network connectivity [9]. This diversity can hinder the effectiveness and efficiency of the FL process, as models must be trained and updated across a broad spectrum of device capabilities.

To address these challenges, lightweight FL has emerged as a promising solution [10]. Lightweight FL aims to optimize the FL process to accommodate the diverse computational resources of participating devices [11]. This involves developing algorithms and protocols that are not only efficient in communication and computation but also adaptive to the varying capabilities of devices [12]. By reducing the computational and communication overhead, lightweight FL ensures that even devices with limited resources can contribute effectively to the learning process without sacrificing performance or accuracy [13].

One popular approach for achieving ML on edge devices is Tiny Machine Learning (TinyML) [14]. TinyML aims to integrate ML algorithms into embedded systems and IoT devices, facilitating data processing as near to the data source as feasible [15]. TinyML offers several benefits:

- **Reduced latency:** by enabling devices to make decisions locally, TinyML minimizes the delay between data generation and data processing, which is crucial for real-time applications.
- **Improved energy efficiency:** local computation is generally less power-intensive than transmitting data to remote servers.
- **Enhanced privacy and security:** since data is processed on the device, there is no need to send sensitive information to a remote server, thereby reducing the risk of data breaches.

Various techniques are associated with TinyML, as illustrated in Fig. 2, though an in-depth discussion is out of the scope of our survey. However, a significant drawback of TinyML is that deep learning (DL) models are difficult to train on-device due to the strict memory, computational, and energy constraints of IoT units. Instead, inference is performed on-device. Fig. 3 depicts the standard TinyML operational workflow, taking the Tensorflow suite as an example. The process starts with training an initial model using a DL framework such as TensorFlow [16]. Subsequently, the TensorFlow model is converted into a TensorFlow Lite (TFLite) [17] model, changing the SavedModel format into the more compact with the format of `.tflite`. The resulting TFLite model is then deployed to the device for real-time

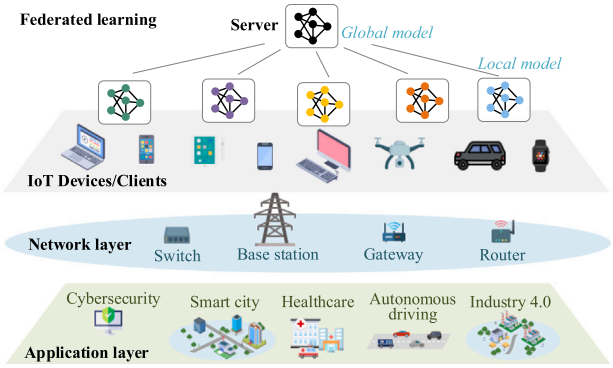


Fig. 1. The application of FL in the IoT spans diverse fields, including healthcare, Industry 4.0, and autonomous driving. The network layer facilitates communication support for the exchange of model updates in FL.

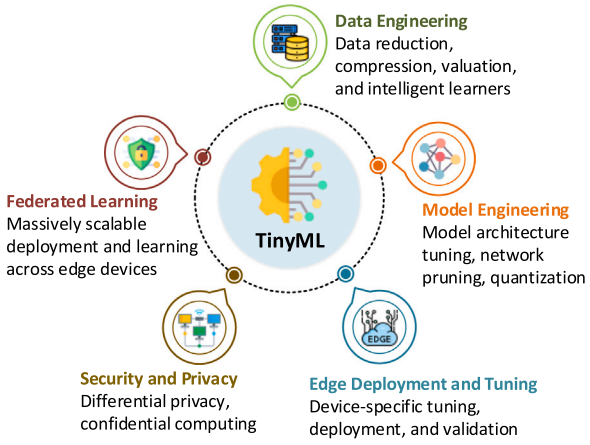


Fig. 2. Technologies and methods related to TinyML.

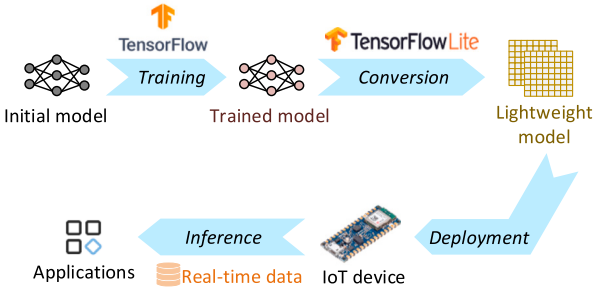


Fig. 3. The training and deployment process of TinyML.

prediction or classification. This established procedure ensures the effective operation of DL models.

Numerous studies have explored effective and lightweight FL techniques that operate with substantially less memory and bandwidth, inspired by TinyML [18,19]. In this work, we also refer to the methods that combine TinyML and FL as TinyFL, as detailed in [20]. By pooling updated model parameters from edge devices, TinyFL facilitates on-device training in memory-constrained environments [21].

Table 1
Comparison of existing surveys on TinyFL and our contribution.

Paper	Year	TinyML	FL	IoT/Edge	Key contribution
[23]	2020	✗	✓	✓	This survey introduces the background of FL and explores its applications in mobile edge networks.
[24]	2021	✓	✗	✓	This survey provides background information on TinyML and also introduces the role of 5G in TinyML-IoT scenarios.
[25]	2022	✓	✗	✓	This survey defines TinyML, outlines its benefits and uses, and explores its integration with network technologies like 5G and Low Power Wide Area Network (LPWAN).
[26]	2022	✓	✗	✓	This survey discusses the key challenges in the embedded vision for IoT based on TinyML.
[27]	2022	✓	✗	✓	This review focuses on the challenges of implementing TinyML on resource-constrained microcontrollers.
[28]	2023	✗	✓	✓	This survey examines the computational heterogeneity among participating devices in FL.
[29]	2023	✓	✗	✓	This survey explores the deployment scenarios of TinyML in extreme edge environments.
[30]	2023	✗	✓	✓	This survey examines the latest advancements in resource optimization for FL within IoT scenarios.
[31]	2023	✗	✓	✓	This survey explores FL in IoT, emphasizing methods for client selection.
Ours		✓	✓	✓	This survey provides insights into how lightweight FL can be implemented to achieve robust and scalable ML solutions in diverse and resource-constrained settings.

This lightweight FL approach ensures model performance while conserving communication resources. In other words, TinyFL represents a subset of ML that focuses on the development of models suitable for execution on low-power, resource-constrained devices [19]. It brings the power of ML to microcontrollers and edge devices, enabling real-time data processing and analytics in environments where traditional, resource-intensive ML models are impractical [22].

1.1. Motivation and contribution

This survey delves into the intricacies of FL, with a particular focus on lightweight FL. We explore the methodologies and technologies that enable efficient FL in heterogeneous environments, highlighting key advancements and ongoing research efforts aimed at overcoming the challenges posed by device heterogeneity. Through a comprehensive analysis, we aim to provide insights into how lightweight FL can be implemented to achieve robust and scalable ML solutions in diverse and resource-constrained settings.

Table 1 compares our survey with related studies. While existing reviews typically target resource-constrained devices such as IoT or edge devices, they generally focus on either TinyML or general FL separately. None comprehensively address the efficient operation and deployment of ML models in these constrained environments: the integration of TinyML and FL presents a promising solution to this challenge. To bridge this gap, we conducted our survey, meticulously collecting and analyzing all existing papers that discuss both FL and TinyML. Although this body of work is currently limited, it has experienced significant growth in the past year, highlighting the increasing recognition of the benefits of integrating these fields. By consolidating these studies, we aim to provide valuable information for practitioners, fostering further advancements and innovation in this emerging area. Our survey highlights pioneering efforts and future directions in lightweight FL, setting the stage for continued progress in this critical domain.

1.2. Survey structure

This survey is structured as follows: Section 2 provides a detailed account of the literature collection process, including search terms, eligibility criteria, and the evaluation methodology. In Section 3, we systematically investigate the lightweight FL workflow, covering aspects from model training to communication and aggregation. We also explore the feasibility of deploying lightweight FL in IoT/edge environments, discussing necessary hardware, software, and more. In Section 4, we present open-source code and tools to promote the reproduction of relevant research on lightweight FL. Finally, in Section 5, we discuss current challenges and propose future research directions. Table 2 lists the abbreviations used throughout this survey.

Table 2
The abbreviations involved in this survey.

FL	Federated Learning
ML	Machine Learning
TinyML	Tiny Machine Learning
DL	Deep Learning
IoT	Internet of Things
IIoT	Industrial Internet of Things
IoMT	Internet of Medical Things
IoV	Internet of Vehicles
TensorFlow Lite	TFLite
TFF	TensorFlow Federated
MCUs	Microcontroller Units
SRAM	Static Random Access Memory
CNNs	Convolutional Neural Networks
RNNs	Recurrent Neural Networks
NLP	Natural Language Processing
VAE	Variational Autoencoder
PTQ	Post-training Quantization
QAT	Quantization-aware Training
SBCs	Single Board Computers
IID	Independent and Identically Distributed
UAVs	Unmanned Aerial Vehicles

2. Literature collection

By adhering to the PRISMA guidelines [32], we ensure that this survey is conducted in a scientifically rigorous manner suitable for a systematic review.

This survey identified two pivotal databases: *Scopus* and *arXiv* to collect current knowledge from various sources. *Scopus* provides an extensive collection of peer-reviewed journal articles and conference papers in computer science and ML. In contrast, *arXiv* offers a repository of preprints yet to be formally published. The search in both databases employed a specific set of keywords: `TITLE-ABS-KEY: ((''lightweight federated'' OR ''tiny federated'' OR ''tinyML'' OR ''tiny'') AND (''federated learning'') AND NOT (''tiny-imagenet'' OR ''tiny Shakespeare''))`, it consists of a combination of these terms (including their plural forms). To avoid increasing the workload of article screening, we excluded keywords related to Tiny ImageNet and Tiny Shakespeare, as they are often mentioned in irrelevant literature.

In addition, we formulated the inclusion and exclusion criteria for the literature according to the research topic of this survey:

- **Inclusion**
  - Research demonstrates FL specifically tailored for IoT/edge environments;

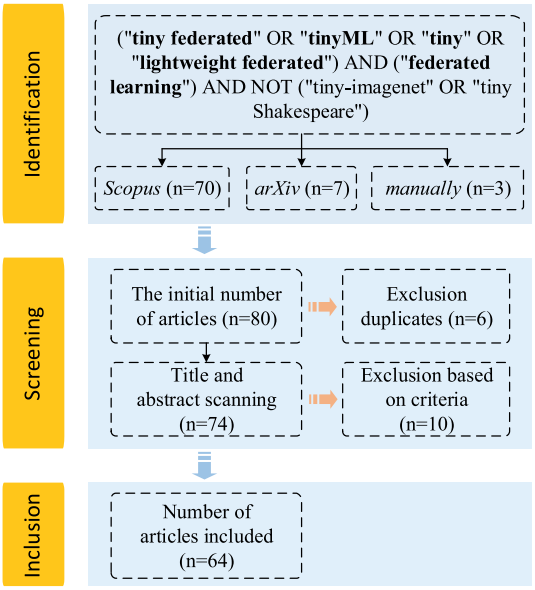


Fig. 4. The literature review in this study follows a systematic article retrieval methodology comprising three distinct stages: identification, screening, and inclusion.

- Research proposes methodologies or techniques specifically designed for lightweight FL;
- Research focuses on aspects of training and inference optimized for resource-constrained devices, such as IoT or edge devices.

• **Exclusion**

- Review papers that summarize existing literature without presenting new methodologies or findings;
- Papers that do not directly contribute to the field of lightweight FL;
- Papers that only mention "Tiny" but do not align their research with the principles or applications of lightweight FL.

The literature screening flowchart, illustrated in Fig. 4, outlines a transparent and reproducible method for reporting the search and selection process. Initially, the search strategy was executed across multiple databases, retrieving 80 papers. Among these, 70 papers were sourced from *Scopus*, 7 from *arXiv*, and additional 3 related papers were manually added to ensure comprehensiveness. Following the initial collection, a deduplication process was conducted, removing 6 duplicate papers. This step yielded 74 unique articles eligible for subsequent screening. Each of these 74 articles underwent meticulous evaluation against predefined eligibility criteria, with a thorough review of their titles and abstracts. As a result, 10 articles that did not meet the required standards or were identified as review-type articles were excluded. This meticulous screening process culminated in selecting 64 articles for an in-depth full-text review, ensuring that only the most pertinent and high-quality studies were considered for further analysis.

Fig. 5 illustrates the annual distribution of the included papers. There has been a steady increase in the number of relevant papers each year, with a notable surge in 2023. By mid-2024, the count of papers was already approaching the total from the previous year, indicating a growing interest and activity in the field.

3. Lightweight FL workflow

Fig. 6 illustrates the lightweight FL training pipeline. While the overall procedure resembles the standard FL, it is constrained by the limited memory and computing power of edge and IoT devices [33]. These limitations are especially evident in devices with the smallest CPU-based systems and highly resource-constrained microcontroller units (MCUs). Some efforts have been undertaken to explore the development of FL in these severely restricted IoT environments, where computing power and storage capacity are at a premium. In this context, innovative methods are needed to overcome these constraints, enabling practitioners to develop and deploy local FL models on IoT devices. Key strategies include reducing the size of the models stored on the devices, enhancing inference performance, and minimizing Static Random Access Memory (SRAM) usage during execution. These approaches are crucial for optimizing FL in resource-constrained environments, ensuring the effective deployment and operation of FL models despite the hardware limitations of IoT devices. The following subsections will detail the model training and inference process.

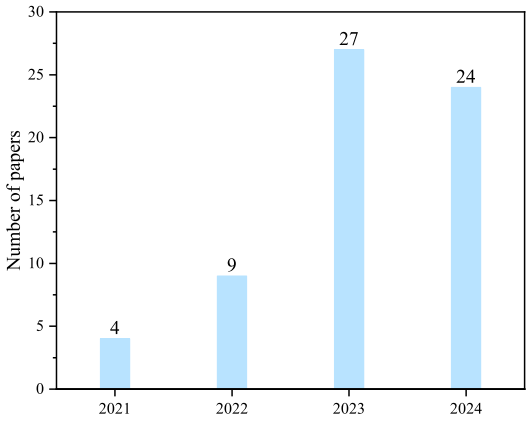


Fig. 5. The number of included papers per year.

3.1. Local model deployment

In lightweight FL, model compression is essential. It accelerates training and inference by decreasing the number of model parameters, significantly cutting storage needs and energy use, thus enhancing system efficiency and adaptability [34]. Key model compression techniques include pruning, quantization, and knowledge distillation, which collectively facilitate the extensive deployment of lightweight FL in resource-limited environments [35].

• Pruning

Model pruning primarily involves two techniques: structured pruning and semi-structured pruning [36]. In structured pruning, some layers of the model are removed, while in semi-structured pruning, specific weights within the layers are set to zero. Previous research has focused on applying pruning to develop sparse FL models for IoT devices [37,38]. However, the pruning process is often memory- and computation-intensive, posing challenges for devices with limited resources. To overcome this, [39] introduces FedTiny, a method designed to create customized, miniaturized models for deployment on devices with constrained memory and processing power. FedTiny uses a lightweight progressive pruning module combined with adaptive search to determine the optimal pruning strategy for each layer incrementally. Experimental results demonstrate that FedTiny achieves a 94.01% reduction in memory usage and a 95.91% reduction in computational cost. FedPARL [40] is a lightweight model that reduces model size through sample-based pruning, ensures client integrity via trust score verification, and optimizes resource usage for improved convergence in IoT environments. Authors demonstrate that tuning pruning parameters enhances FL's robustness.

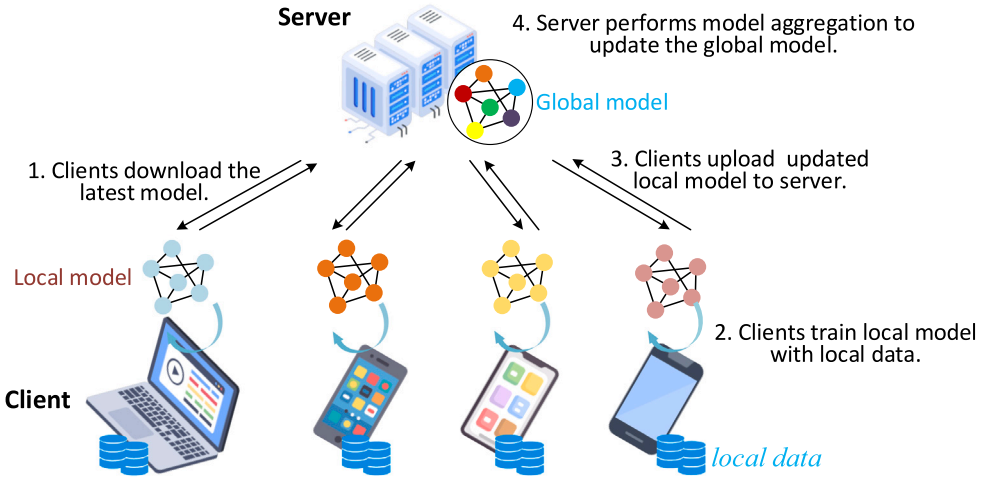


Fig. 6. The pipeline of lightweight FL training.

- Quantization
  Typically, deep neural network weights are stored as 32-bit floating point numbers. Model quantization often converts these weights to 16-bit, 8-bit, or even 4-bit integers. Depending on the specific procedures used, quantization can be classified into post-training quantization (PTQ) and quantization-aware training (QAT) [41]. In particular, PTQ involves converting a pre-trained model into a quantized version without additional training, making it cost-effective. This method is particularly suitable for compressing large models to sizes appropriate for deployment on IoT devices. For instance, [42] introduces TinyOFL, a method leveraging TinyML devices for online FL. This approach employs quantization and dequantization for model communication on both the clients and the server, as illustrated in Fig. 7. The findings demonstrate that TinyOFL can reduce power consumption and communication bandwidth while preserving an accuracy level comparable to that of a full-precision model. TinyFedTL, as introduced in [43], utilizes transfer learning to compress a pre-trained MobileNet model by a factor of 0.35. This compressed model is then frozen, and its inputs and outputs are quantized to 8-bit using PTQ to support MCUs. This approach results in significant efficiency gains in terms of memory usage (Flash, SRAM) and eliminates the need for frequent communication with a central server, making it highly suitable for IoT hardware. However, research conducted in [44] has demonstrated that directly integrating advanced INT8 (8-bit integer) training algorithms with traditional FL protocols can lead to a notable reduction in model accuracy. This decrease is primarily attributed to the computational burden imposed by frequent quantization procedures on DSP-enabled INT8 training. To address this issue, [44] introduced Q-FedUpdate, which employs a pipelining technique to reduce floating-point computational overhead by parallelizing CPU-based quantization with DSP-enabled training.
- Knowledge distillation
  Knowledge distillation is a technique that transfers knowledge from a large model (called teacher) to a smaller model (called student) [45]. This process enables the student model to achieve nearly the same performance as the teacher model while using fewer processing resources. By training the student model with soft labels generated by the teacher model, knowledge distillation effectively compresses the model and speeds up inference without significantly sacrificing performance [46]. For instance, in [47], the Fed2KD framework is introduced, which utilizes bidirectional knowledge distillation to facilitate the exchange of knowledge between server and clients. This process involves distilling information both into and from small models that share unified configurations. To address the challenge of requiring a common dataset for distillation, the framework incorporates a conditional variational Autoencoder (cVAE) to generate synthetic data, which serves as a substitute dataset for the distillation process. However, it is crucial to consider that computing the difference in output distribution between teacher and student models increases the computational load during training [48]. This can be particularly challenging for resource-constrained edge devices.

3.2. Local model training

Microcontrollers face severe limitations in terms of power, memory, and storage capacity, with memory sizes that can be up to 50,000 times smaller than those found in GPUs [49]. These constraints necessitate extremely compact models for efficient operation on such devices. Researchers have focused on enhancing the training process for TinyFL by developing strategies that optimize model training under these constraints.

On the one hand, it is effective to deploy low-cost ML models on resource-constrained devices. These models, while not delivering the highest performance, have certain advantages during the training phase due to their simple architecture and fewer parameters. For instance, [50] employs Bayesian classifiers as models in FL clients. These micro Bayesian classifiers handle only small training datasets for knowledge acquisition, making them easy to deploy at the edge or device level due to their low cost and reduced parameter count.

Alternatively, another approach involves partitioning a complete model into smaller sub-models distributed across multiple devices for joint training. FedDCT, proposed in [51], divides a large model into several sub-models that are trained in parallel on different devices. After training, these sub-models are merged to form a complete global model. While effective for models like Convolutional Neural Networks (CNNs), this approach may not be suitable for models requiring temporal dependencies, such as Recurrent Neural Networks (RNNs) [51].

Another promising strategy is to use a large model deployed on the large device to guide a tiny model trained on an edge device. This method offloads the training burden of high-performance models to powerful participants, allowing resource-constrained micro devices to focus solely on training micro models. In FedTinyAug [52], the server leverages the additional capacity of larger participating devices to build an improved model during training. This enhanced model is then distributed to the larger devices to provide supplementary supervision for training the micromodels.

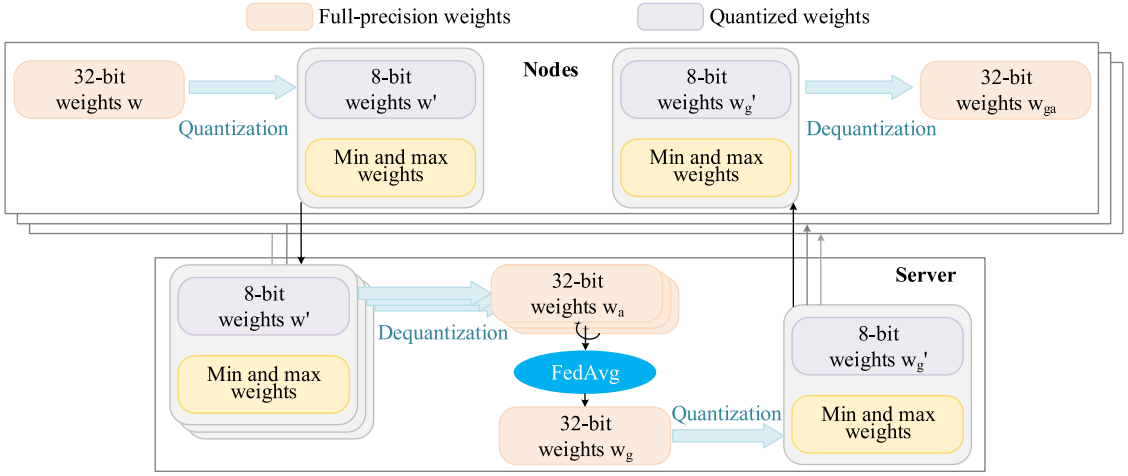


Fig. 7. Quantization and dequantization on the clients and server in FL process, from research work [42]. Client nodes convert their local weights from floating-point to integers, sending the transformation metadata (including *max* and *min* weights) to the server. The server uses this metadata to dequantize the received 8-bit integers back to floating-point weights, applies FedAvg to obtain the global model weights w_g, and then quantizes the updated weights before distributing the new model back to the clients.

3.3. Server–client communication strategies

Communication bottlenecks are increasingly becoming major obstacles to the widespread application of FL, significantly hindering its large-scale deployment [53]. During the training phase, FL requires frequent transmission of model parameters between participating clients and a server. This process is crucial for the collaborative improvement of the model across various nodes. However, the network connection rate and bandwidth available to these clients are usually limited, leading to slow and inefficient data transfers [34]. These constraints prolong the training process and raise the communication overhead due to the high frequency of model exchange required. In environments where clients are distributed across regions with varying network capabilities, the challenge becomes even more pronounced. Limited bandwidth can cause bottlenecks, resulting in delays and reduced overall system performance. Addressing these communication challenges is essential for FL's flexible and scalable implementation. In addition to adopting compression techniques or developing more efficient algorithms introduced in the previous section, optimizing data transmission protocols and improving communication strategy solutions are essential to overcoming these obstacles. In [54], the authors propose online FL algorithms designed to substantially reduce communication overhead. They introduce PSO-Fed methods, which aim to decrease communication costs by transmitting only a subset of model parameters during each aggregation round. Additionally, to further minimize communication expenses, clients are enabled to engage in local learning and share models only when newly available input data is deemed sufficiently innovative. This selective approach ensures that communication occurs only when essential, thereby optimizing the efficiency of the FL process.

Network protocols have evolved to meet contemporary communication needs and trends. In the context of FL, especially within wireless networks [55], specialized network protocols, and methods have been developed to tackle FL-specific challenges, such as balancing resource scheduling and ensuring seamless communication [56]. One major challenge in FL is network resource scheduling, due to client heterogeneity and bandwidth limitations [57]. Many FL research frequently employs static workload partitioning, which is not well-suited for dynamic distributed IoT environments [58]. Effective FL deployment requires careful consideration of client distribution, network condition, and communication capacity [59]. The server must also coordinate which clients participate in the training and updating processes. For example, the Hybrid-FL [60] communication protocol addresses these issues by considering the data distribution and channel conditions of each client to ensure smooth communication. In this protocol, the client reports its data volume, computing power, and communication conditions to the server. This information helps estimate the time required for the update, and then the selected client transmits the model to perform the update. Authors in [34] introduce a communication-efficient FL approach by constructing a small synthetic dataset with synthetic features derived from the original gradient, aiming to optimize the communication efficiency of the FL process. Eco-F, introduced by [61], employs adaptive scheduling to adjust task distribution and client grouping across different levels. Training workers regularly report their execution times to the portal node. If the portal node detects a significant discrepancy between a device's current and historical execution times, it adaptively reschedules the pipeline, updating the device order and workload division points. Once the workload migration is complete, the portal node instructs the head node to resume training.

3.4. Global model aggregation

Model aggregation is an important aspect of FL, involving the combination of local models on client devices into a global model. The determination of the aggregation method is critical, as it directly impacts the accuracy and reliability of the global model. lightweight FL, where models are reduced in scale and depth to accommodate constrained environmental conditions. Choosing suitable aggregation strategies, and ensuring that the performance of the global model remains stable or is only minimally affected. Previous research has categorized techniques of FL aggregation into 4 types based on the aggregation form: synchronous aggregation, asynchronous aggregation, hierarchical aggregation, and robust aggregation [62]. These different aggregation methods offer different benefits and trade-offs, making it crucial to choose the appropriate technique based on the specific requirements and constraints of the lightweight FL environment.

Synchronous aggregation in FL requires aggregation to occur only after the server has received all client updates [51]. Fig. 8(a) shows the synchronous aggregation process in FL. This method typically overlooks the delays or lags experienced by individual clients. While this approach is common in theoretical FL research, it does not align well with real-world IoT environments where devices may have varying computing power and communication capabilities. To optimize communication throughput and enhance global models, some studies have proposed advanced aggregation methods based on synchronous aggregation [63]. This is particularly important for FL in IoT environments, where efficient communication is crucial. In [64], after receiving a model from each client, the server assesses the communication throughput between

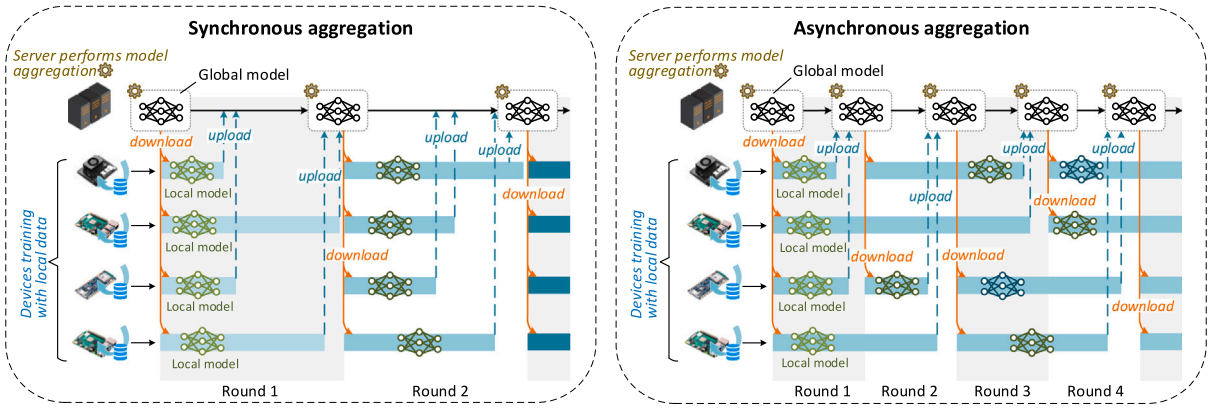


Fig. 8. (a) Synchronous aggregation for lightweight FL, where the server performs aggregation only after receiving updates from all clients. (b) Asynchronous aggregation for lightweight FL, where the server can start aggregation after receiving updates from a subset of clients.

itself and each client. Based on this evaluation, the server adaptively selects clients for aggregation to achieve the optimal structure of the global model.

Asynchronous aggregation enables clients to upload their local updates in a staggered manner, addressing the issue of device heterogeneity [65]. Fig. 8(b) shows the asynchronous aggregation process in FL. In this approach, aggregation occurs when the server receives updates from the client, enabling each client to train independently without waiting for others. To ensure client data security in FL without relying on complex configurations and environments, and to maintain scalability, [66] employs a one-way active communication method. This framework allows clients to actively access the server asynchronously, enabling multiple clients to connect and send models at different times. This approach constitutes an asynchronous aggregation strategy. [67] introduces FLight, a lightweight FL framework for resource-constrained edge devices. Comparing FLight's lightweight worker selection approach to sequential training, it takes 34% less training time to obtain 80% accuracy. Its asynchronous methodology also reduces the synchronous FL training time by 64%.

Hierarchical aggregation introduces an edge layer that partially aggregates local models from closely related client devices before further aggregation on a cloud server [68,69]. This method aims to decrease communication overhead and the number of model transmission rounds by utilizing multiple aggregation centers, thereby optimizing the overall aggregation process [70]. In [61], an adaptive scheduling strategy is introduced to address resource fluctuation issues in federated systems running on resource-constrained devices. This strategy incorporates a hierarchical model aggregation architecture and a dynamic grouping approach that considers device heterogeneity. In [71], LightFIDS is presented as a lightweight FL framework featuring a hierarchical structure. Clients operate at the base level, cluster heads manage client groups at an intermediate level, and cluster master nodes aggregate updates from cluster heads to form global model updates. The study demonstrates that LightFIDS achieves superior performance over traditional FL frameworks in terms of communication cost, convergence speed, and scalability. Furthermore, [72] introduces a method for deploying and optimizing lightweight FL tasks in open wireless access networks for 5G applications. This method employs a 'client-edge-cloud' architecture with three layers, facilitating local model updates at the 'client-edge' layer and global aggregation between the 'edge-cloud' layer. The proposed scheme leverages reinforcement learning for client selection and resource allocation, followed by network slicing based on the selected client for each FL task, ensuring efficient model training in each round.

Robust aggregation addresses the security risks potential in FL, such as backdoor attacks, inference attacks, and zero-day vulnerabilities [73]. To mitigate these risks, techniques such as differential privacy and homomorphic encryption are employed to address security vulnerabilities and enhance system privacy protection. To enhance privacy protection in FL, it is crucial to encrypt the gradient updates of model parameters exchanged between servers and participants, as these updates can be vulnerable to attacks [33]. In [74], FLOGCNN is introduced as a lightweight FL anomaly detection technique that utilizes distributed log data. In this framework, FLOGCNN servers aggregate gradient updates based on participant sample sizes to construct an integrated model. In [75], the challenge of limited resources in edge computing environments is addressed by employing mask and secret sharing techniques for secure aggregation of client models. This method significantly reduces the system resources required for model training. Moreover, the technique evaluates each client's contribution by assessing data similarity between clients, which helps in defending against attacks from malicious participants. Furthermore, [10] presents a lightweight, privacy-preserving FedL scheme for IoT devices by integrating masks into intermediate parameters. An efficient secret-sharing scheme ensures the accurate removal of these masks, demonstrating robust security against honest-but-curious adversaries.

3.5. Evaluation metrics

In lightweight FL, optimizing performance and efficiency is crucial, particularly when deploying on resource-constrained devices like IoT devices and mobile phones.

(A) **Performance** The performance of FL models is commonly assessed using several key metrics:

- Accuracy: This refers to how well the final model performs on test data [76]. Common metrics for evaluating accuracy include precision, recall, and the F1 score [74]. When optimizing the model for deployment on lightweight devices, it is essential to maintain high accuracy even if adjustments are made to reduce the model's size. The model must remain both efficient and effective in fulfilling its intended purpose.
- Convergence Speed: This metric measures how quickly the model reaches its expected performance level [20]. It is typically assessed by counting the number of communication rounds required for the model to converge [72]. Faster convergence is desirable because it implies the model can achieve satisfactory performance with fewer communication rounds. If the convergence speed is too slow, it indicates that the model is not performing efficiently, leading to prolonged training times and potentially inadequate results.

(B) **Efficiency** In addition to performance, communication efficiency and energy consumption are critical factors in lightweight FL:

- Communication Overhead: This refers to the volume of data exchanged between devices and servers during the training process [74]. Minimizing communication overhead is vital, especially in bandwidth-limited environments, as it directly impacts the efficiency and speed of the learning process.
- Energy Consumption: Energy efficiency is particularly important for battery-powered devices involved in FL [42]. Since many edge devices have limited battery capacity, excessive energy consumption can cause these devices to deplete their batteries rapidly [77]. This could lead to their premature exit from the FL training process, compromising the overall operation and stability of the FL system.

In summary, when implementing lightweight FL, it is essential to balance high model accuracy, fast convergence, low communication overhead, and minimal energy consumption to ensure both effective and efficient performance on resource-constrained devices.

3.6. Deployment and application

Among the many real-world applications of lightweight FL, several notable examples demonstrate its versatility and effectiveness across different sectors. The contemporary IoT, encompassing domains such as the Internet of Medical Things (IoMT), Industrial Internet of Things (IIoT), and Internet of Vehicles (IoV), has significantly contributed to the advancement of modern society [78]. IoT enables numerous smart applications by providing a sensing layer where IoT devices collect data at the network's edges [79]. This data is transmitted over broadband internet to higher-level nodes for analysis and inference, which drives the entire IoT ecosystem.

3.6.1. IoT

A notable area of interest is IoT, where [80] propose a lightweight mini-batch FL method for detecting network attacks in IoT environments. Results show that the proposed approach achieves a 98.85% attack detection accuracy, with a false alarm rate of just 0.09%, requiring fewer federation rounds than traditional FL methods to converge, thus effectively detecting attacks with minimal resource usage. Another study [81] addresses IoT challenges by adding privacy-preserving masks to data parameters and using a secret-sharing technique, further optimizing FL for IoT applications. Typically, IoT systems incorporate Microcontroller units (MCUs) or Single Board Computers (SBCs) [82]. These devices are highly integrated chips, have limited memory, and are powered by little batteries, and they include a radio transceiver for wireless connectivity [83]. Due to limited memory and power constraints, deploying deep models directly onto these devices for training is usually not feasible. However, efforts are being made to develop deployment architectures that address these limitations [84].

- Microcontroller Units
  MCUs with substantial onboard memory, including both RAM and FLASH, and low power consumption are essential for delivering local computing power in IoT applications [84]. This balance is critical as it allows devices to perform computing tasks independently without constantly relying on external servers, improving response times and reducing latency [85]. Fig. 9 presents a performance comparison of commercial IoT MCUs, as analyzed by [84]. Many current MCUs still fail to fully meet the dual demands of high onboard memory and low power consumption, due to the inherent trade-offs manufacturers face when designing these devices—enhancing memory often comes at the cost of increased power consumption, and vice versa.
- Single Board Computers
  The SBCs [86] are compact and self-contained computing devices with all essential components, like the CPU, memory, input/output interfaces, and storage, on one circuit board. Unlike traditional computers, SBCs do not need extra expansion cards or external modules for basic functions. They use low-power processors (such as ARM [87]) and offer ports like USB, HDMI, GPIO, and Ethernet for connecting peripherals. SBCs can run different operating systems, like Windows, Linux, and Android, making them versatile for many uses [88]. They are popular in fields like IoT, robotics, and industrial automation because they are small, affordable, and expandable [89]. Examples include the Raspberry Pi, BeagleBone, and Arduino [90].

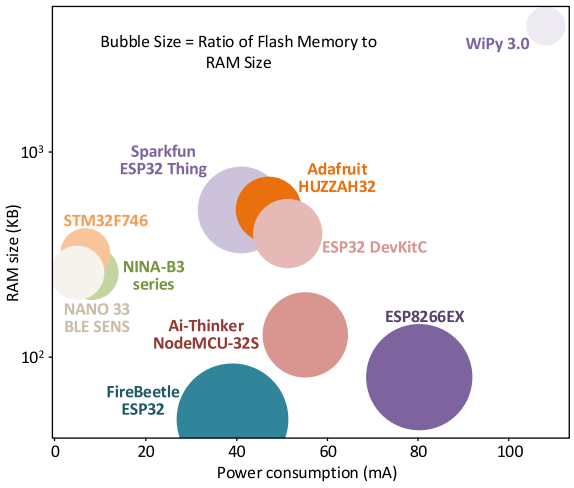


Fig. 9. Power consumption and available memory of commercial IoT MCUs [84].

Table 3
Comparison of characteristics of MCUs and SBCs.

Characteristic	MCUs	SBCs
Processing power	Low	High
Memory and storage	Limited	Rich
Power requirements	Low	High
Application scenarios	Embedded systems, real-time control	Education, development, media, IoT, and multifunctional applications
Common examples	Arduino, STM32	Raspberry Pi, BeagleBone

Table 3 lists the feature comparison between MCUs and SBCs. In summary, MCUs are suitable for specific, low-power, real-time tasks, while SBCs are suitable for applications that require higher processing power and multi-functionality. In addition, Table 4 lists common IoT devices, and their attributes, and summarizes the literature utilizing these devices for lightweight FL training and inference. Notably, the Raspberry Pi 4 Model B stands out for its superior performance and greater computing power.

3.6.2. Healthcare

In the healthcare domain, ensuring the confidentiality of medical data and safeguarding user privacy are critical [96,97]. Recent advancements have explored FL to address these challenges. For instance, researchers in [98–100] proposed lightweight FL architectures for classifying liver cancer and Alzheimer's disease, leveraging datasets with independent and identically distributed (IID) characteristics. Despite achieving high prediction accuracy, the heterogeneous nature of data distribution remains a significant consideration in FL environments [101,102]. [64] represents an application of a lightweight FL-based solution in medicine that prioritizes high privacy protection and enhances communication throughput for predicting the risk of sexually transmissible infections and human immunodeficiency virus. The paper introduces a method for reducing AI model size on the server side, which enhances system throughput, decreases power consumption, and accelerates response times. This makes the model more

Table 4
Common IoT devices, their attributes, and relevant studies.

Device	Attributes	Studies
ESP32	Low-cost, low-power microcontroller with dual-mode Bluetooth and integrated Wi-Fi.	[19]
ESP8266	Low-cost Wi-Fi microcontroller with microcontroller capability and TCP/IP networking software integrated in.	[19,84]
Arduino UNO WiFi Rev2	Featuring a cryptographic chip accelerator, this device uses an ATmega4809 8-bit microcontroller and has an onboard IMU and NINA-W102 module for Wi-Fi and Bluetooth.	[19]
Arduino MKR WiFi 1010	It uses the Arm® Cortex®-M0 32-bit SAMD21 processor and is also equipped with an ECC508 encryption chip.	[19]
Arduino Portenta H7	Portenta H7 features dual processors capable of running high-level code and real-time tasks in parallel.	[42,77,91]
Arduino Nano BLE 33	Based on the nRF52840 microcontroller with 1 MB CPU Flash, it features a 9-axis IMU and Bluetooth Low Energy connectivity.	[92,93]
Raspberry Pi Zero W	It adds Bluetooth and wireless LAN connection, extending the Pi Zero family.	[19]
Raspberry Pi 2 Model B	It sports a 1 GB RAM and a 900 MHz quad-core ARM Cortex-A7 CPU.	[72]
Raspberry Pi 3 Model B/B+	It has dual-band Wi-Fi, Bluetooth 4.2/BLE, a 1.4 GHz 64-bit quad-core processor, and faster Ethernet.	[19]
Raspberry Pi 4 Model B	It features a 8 GB RAM, Wi-Fi, Bluetooth, full Gigabit Ethernet, and a 1.5 GHz quad-core ARM Cortex-A72 processor.	[79,92–95]
Nvidia Jetson Nano	It features a quad-core 64-bit ARM CPU, a 128-core NVIDIA GPU delivering 472 GFLOPS, and 4GB of LPDDR4 memory in a low-power package.	[61,72]
Nvidia Jetson TX2	It boasts a compact card design, making it ideal for low-power applications, such as small camera drones.	[61]

suitable for mobile and edge devices, prioritizing privacy and efficient communication. A notable approach by [103] introduced a lightweight FL method for COVID-19 detection using chest CT images. Their method involves clients uploading aggregated feature vectors instead of raw model parameters during training, enhancing data privacy while reducing communication overhead. Moreover, clients can adapt local models based on their computational capacities, further optimizing efficiency. The results indicate that even tough accuracy is slightly lower than that of FedAvg, the method offers a substantial reduction in communication overhead compared to it. Similarly, [104] proposed an FL framework enabling multiple hospitals to collaborate on COVID-19 screening using chest X-rays, demonstrating both the feasibility and privacy benefits of FL in medical imaging. In another innovative application, [105] illustrated privacy-preserving FL using an air–ground network system, where Unmanned Aerial Vehicles (UAVs) collect data from wearable devices and environmental sensors of mobile users. This data fuels a collaborative FL model to privately learn medical symptoms such as those related to COVID-19, highlighting the versatility of FL across diverse healthcare scenarios. These advancements underscore the evolving landscape of lightweight FL in healthcare, emphasizing its role in balancing data utility with stringent privacy requirements, thereby advancing both medical research and patient care.

3.6.3. Industry

Given the benefits of lightweight FL, many researchers have applied it to industrial systems, aiming to ensure efficient DL performance while enhancing data privacy. For instance, [106] addressed the challenge of intrusion detection in industrial control systems by evolving Neural Architecture Search (NAS) and designing a lightweight FL model named Fed-GA-CNN-IDS. Their approach leverages the advantages of FL to develop a model that can detect intrusions effectively while maintaining the privacy of sensitive data. Similarly, [107] proposed an improved feature extraction technique specifically for reciprocating machinery using an enhanced federated lightweight network. This method uses a FL framework to address the problem of limited model training due to inadequate data. The framework enables collaborative diagnosis of machinery data across multiple enterprises, ensuring a more robust and accurate analysis. To address the challenge of long training times commonly associated with DL, [107] also utilized the lightweight SqueezeNet model, which offers efficient training and inference without compromising performance. This approach significantly reduces the computational burden and accelerates the training process, making it a practical solution for real-world industrial applications. In particular, the proposed lightweight FL network achieved a fault diagnosis accuracy of 96.6%, with the training time reduced to 1982 s, a decrease of approximately 41.5%, compared to benchmark. [108] introduce an optimized lightweight FL method for botnet attack detection in smart critical infrastructure. The study employs a feature dimensionality reduction technique to minimize the memory required to store network traffic data at edge nodes, enhancing system efficiency and making it well-suited for critical infrastructure protection.

3.6.4. Transportation

Urban transportation infrastructures can greatly expedite the processing of vast and diverse data from multiple sources by integrating edge computing paradigms [109]. Moreover, integrating FL into these infrastructures shows promise in delivering reliable and intelligent transportation services [110]. As shown in Fig. 10, lightweight FL is utilized for intelligent transportation. In this framework, vehicles serve as training agents, base stations function as edge aggregators, and the central server conducts the final aggregation. This hierarchical approach enhances the training of neural networks to predict traffic patterns within the transportation network [111].

In their work, [6] introduce Fed-AVIS (Federated Learning-based Anti-Vehicle Infiltration System), an anti-vehicle infiltration system based on FL. The system utilizes lightweight DL models like TinyYOLOv4 and MobileNetV2 to achieve strong performance while maintaining simplicity. It utilizes Azure service bus queues to minimize alert message delays and operates efficiently on edge devices like Raspberry Pi, demonstrating low power consumption between 2.84 and 4 watts at maximum capacity. Also, the average message delay is 6.5 s, with the longest delay recorded at 11 s and the shortest at 4.

Additionally, [112] presents FedLVR, a lightweight, privacy-preserving FL framework tailored for fine-grained vehicle detection in edge-assisted intelligent transportation systems. The framework introduces a deployable lightweight vehicle recognition model on edge

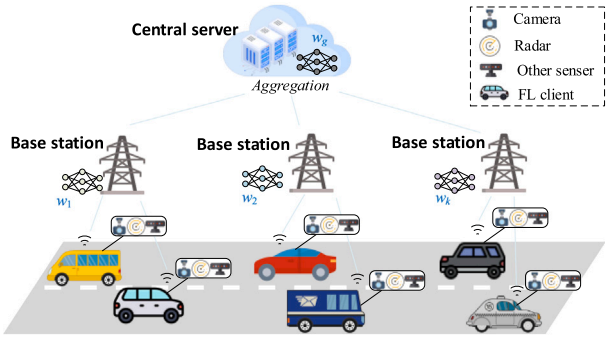


Fig. 10. The lightweight FL used for intelligent transportation. Where onboard cameras, radars, and other sensors gather data on road conditions. Base stations are utilized for model training, while servers aggregate to form a global model.

nodes, achieving high accuracy and significantly reducing model complexity in experimental evaluations.

3.6.5. Natural language processing

Furthermore, lightweight FL is increasingly being integrated into a variety of practical applications, such as Natural Language Processing (NLP). In [113], a keyword spotting application was developed to explore training FL models on an Arduino Nano 33 BLE Sense microcontroller. Users begin by selecting up to three keywords, triggering the on-device ML model to train online. Detected keywords correspondingly illuminate the RGB LED on the Arduino board. During FL training, the locally trained model exchanges data with a central server. Once trained, users can test keyword detection using the trained model by switching to inference mode. The research report that the model's performance improved with training on spoken words, and the loss consistently decreased with more training rounds. This approach demonstrates the feasibility and effectiveness of deploying FL models on resource-constrained MCUs.

In a related study, [42] presents a case study on the TinyOFL framework, advocating for the use of TinyML devices in online FL for IoT applications. Using three Arduino Portenta H7 development boards, the study implements an FL client for a keyword-spotting task. TinyOFL's quasi-batch training capability supports aggressive weight quantization, enhancing efficiency without sacrificing accuracy. The results show that 7-bit quantization of TinyOFL reduces bandwidth by 4.6 times without loss of accuracy, while 5-bit weights achieve 74% accuracy with FL.

Additionally, [77] explores FL on microcontroller boards through a LoRa mesh network. The system employs a dual-board setup: an Arduino Portenta H7 trains a neural network for keyword identification, while a TTGO LORA32 board serves as a router. This configuration enables FL with retrainable nodes, fully deployable via LoRa connectivity. The authors evaluated various aspects of model performance, bandwidth usage, and energy consumption, demonstrating that the proposed approach enhances the accuracy of the local model on the active node while reducing communication costs.

3.6.6. UVA

Research has explored the application of lightweight FL in edge and IoT scenarios, showcasing several notable cases. A significant example involves drones utilized as complementary IoT devices [110,114]. These drones operate at similar altitudes, performing tasks such as data collection and commercial delivery [115,116].

In [117], the EET-SAGIN algorithm is introduced, leveraging the TinyFDRL framework within a sophisticated four-layer aerial computing system. This system integrates ground nodes, low-altitude UAVs, high-altitude UAVs, and satellites. The algorithm seamlessly combines advanced ML techniques including FL, deep reinforcement learning, and TinyML. Operating across diverse environments from urban cities to remote rural areas, it facilitates distributed intelligent UAV applications through collaborative strategies. Simulation results of the research work show that as the number of low-altitude drones increases, energy consumption rises for all methods. However, TinyFDRL consistently has the lowest energy consumption with 20 drones due to optimized parameter learning through FL.

Furthermore, [118] suggests a spatial adaptive playback-based multi-UAV energy-efficient coverage deployment algorithm. This method employs enhanced particle swarm optimization (PSO) to optimize UAV deployment locations considering signal-to-interference ratios. Additionally, a deep deterministic policy gradient method dynamically allocates resources, optimizing energy consumption and link delays between users and the UAV system. This approach enables the effective execution of lightweight FL across multiple iterations. The experimental results show that the adopted technology reduces system costs, decreases FL training rounds by 66.67%, and outperforms other resource allocation methods in minimizing latency and energy consumption.

3.6.7. Sustainable development

In the pursuit of sustainable development, advancements in lightweight FL and TinyML have emerged as pivotal technologies across diverse applications. These innovations not only enhance the efficiency and accuracy of ML tasks but also contribute to resource conservation and environmental sustainability. [94] introduce Lite-Agro 2.0, a pioneering framework aimed at pear disease classification using FL and TinyML. Their setup, comprising Raspberry Pi 4 and 3B models in a Pi Cluster rack, explores various FL frameworks and algorithms to analyze pear diseases through leaf analysis. This approach demonstrates the feasibility of deploying complex ML tasks on resource-constrained edge devices, thereby promoting sustainable agricultural practices.

In parallel studies, [119] focus on tomato disease classification, developing a lightweight federated deep learning architecture. Their work emphasizes preserving data privacy while achieving robust classification performance for tomato-related diseases. The experimental results indicate that, among the seven pre-trained networks tested, AlexNet achieved the highest accuracy rate at 99.59%. Similarly, [120] presents a Lightweight Federated Transfer Learning (LFTL) framework designed to improve leaf disease identification. The LFTL framework surpasses traditional distributed deep learning methods in scenarios with both independent and non-independent datasets, showcasing its adaptability and efficiency in agricultural settings.

[76] extend the application of lightweight FL to rice leaf disease classification and detection, emphasizing the role of FL techniques in enhancing agricultural productivity through accurate disease diagnosis, thereby helping to minimize crop losses. In addition, the research report shows that the pre-trained model EfficientNetB3 they used helped the results achieve the highest training and evaluation accuracy of 99%. Meanwhile, [121] investigates the use of lightweight FL in smart energy meter data analysis for load forecasting. The authors demonstrate that by leveraging a lightweight fully connected deep neural network within the FL framework, they can achieve forecasting accuracy comparable to state-of-the-art methods while preserving the privacy of individual meter data. This approach is particularly well-suited for resource-constrained environments, like smart meter systems, due to its reduced energy and resource consumption.

Finally, [122] proposes deploying deep learning models on mobile devices using a lightweight FL model. By optimizing spectrum and communication resources through collaborative training by edge servers and mobile terminals, they achieve notable energy savings and reduced processing latency. This advancement is pivotal for developing intelligent applications that operate efficiently on mobile platforms while maintaining robust performance standards. Together, these studies underscore the transformative potential of lightweight FL and TinyML in advancing sustainable development goals across agriculture, energy management, and mobile computing sectors.

Table 5
Common federated learning and edge computing platforms.

Function	Platform	Link
Federated learning	TFF	https://www.tensorflow.org/federated
	PySyft	https://blog.openmined.org/tag/pysyft/
	FedML	https://www.fedml.ai/
	Flower	https://flower.ai/
	FATE	https://fate.readthedocs.io/en/latest/
Edge computing	TensorFlow Lite	https://www.tensorflow.org/lite
	Azure IoT Edge	https://azure.microsoft.com/it-it/products/iot-edge
	KubeEdge	https://kubeedge.io/
	AWS Greengrass	https://aws.amazon.com/it/greengrass/
	EdgeX Foundry	https://www.edgexfoundry.org/

4. Open source tools and code

Deploying lightweight FL to edge networks, particularly in IoT environments, requires robust software infrastructure to guarantee smooth operations across various stages of the ML lifecycle [22]. This includes essential support for efficient model training, seamless distribution of models to edge devices, timely updates to models and software, and effective communication protocols to maintain synchronization and data integrity.

4.1. Software platform

On the one hand, there are several prominent FL frameworks, each serving a vital role in facilitating FL on distributed devices. These frameworks include TensorFlow Federated (TFF) [123], PySyft [124], FedML [125], Flower [126], and FATE [127], collectively advancing the capabilities of decentralized ML. On the other hand, in the realm of edge computing, there exist versatile ML frameworks tailored for lower-capacity hardware, pivotal in shifting computational processes from centralized cloud environments to the network's edge. Noteworthy examples of such frameworks are TensorFlow Lite [17], Azure IoT Edge [128], KubeEdge [129], AWS Greengrass [130], and EdgeX Foundry [131], each designed to enhance efficiency and performance in edge computing scenarios. Table 5 summarizes the access methods for these widely used ML platforms.

4.2. Open source code

To facilitate the effective deployment of lightweight FL, Table 6 compiles links to relevant open-source projects in the collected papers, aiding replication and serving as a reference for future research endeavors.

5. Challenges and future directions

The combination of TinyML and FL holds great promise, particularly in distributed scenarios like IoT, where device resources are constrained [95]. FL ensures robust privacy protections for local data and promotes the development of secure distributed systems [133]. Nevertheless, several challenges currently impede the widespread adoption of lightweight FL. Addressing these challenges is anticipated to unlock significant developmental opportunities in this field.

5.1. Device heterogeneity

The IoT ecosystem is characterized by a vast array of devices that exhibit diversity in terms of performance capabilities and operational environments [134]. This inherent heterogeneity presents substantial challenges in the unified management and coordination of these devices [135]. One of the primary difficulties lies in ensuring synchronization and consistency during the training processes across such varied devices [136]. The complex and varied nature of the operating environments further complicates this task, as each IoT device operates under different conditions and constraints. Moreover, the diverse environments where IoT devices are deployed result in the collection of heterogeneous data [137]. This variability in locally collected data can lead to discrepancies in local model performance [132].

To address these challenges, FL multi-task learning strategies offer a promising approach [138]. These strategies enable IoT devices with similar attributes to collaboratively develop and customize models. By grouping devices based on their shared characteristics, it becomes possible to tailor models optimized for each group's specific resources and application requirements [61]. This customization ensures that devices can operate efficiently within their unique environments while maintaining high performance.

5.2. Energy consumption

During the FL phase, IoT devices typically engage in frequent communication with a central server [139]. This interaction is crucial for tasks such as model aggregation, updates, and synchronization. Depending on the specific aggregation method employed, model parameters may need to be exchanged multiple times between the client (IoT device) and the server. This frequent exchange places a significant strain on resource-constrained devices [140].

Although IoT devices play a pivotal role in the development of ubiquitous computing and smart environments [141], their small size and ease of deployment present both advantages and challenges. Small batteries predominantly power IoT devices, and the high energy consumption required for computing and communication tasks accelerates battery depletion [142]. This rapid power consumption can severely impact the normal operation and service life of these devices. Therefore, energy efficiency is a critical concern in the practical application of lightweight FL.

To address this issue, it is essential to enhance the training and aggregation strategies of FL models to reduce their power consumption. Additionally, developers must collaborate to design low-power, high-RAM IoT devices, reflecting a significant industry requirement and trend [143].

5.3. Security concerns

Lightweight FL, while inherently reducing some security risks by not requiring the transmission of raw data to a central server, still faces significant security challenges similar to those in general FL systems. The primary security concerns in this context revolve around poisoning attacks and reconstruction attacks, both of which can compromise the integrity and privacy of the distributed learning process.

Poisoning attacks involve adversaries injecting malicious data into the training process, leading to the degradation of local model performance. When these compromised models are aggregated, the entire FL system can be adversely affected. In Section 3.4, we introduce several secure aggregation techniques such as differential privacy, homomorphic encryption, and multi-party secure computing, which are designed to mitigate these risks. However, while these methods enhance security, they also introduce additional computational overhead, which is particularly challenging for the resource-constrained devices often used in lightweight FL. In reconstruction attacks, adversaries attempt to reconstruct the original input data from gradient information or model parameters. This risk is particularly high if the central FL server is compromised or malicious. Protecting against such attacks requires sophisticated encryption and obfuscation techniques, but these, too, can increase computational demands, which is a critical concern in lightweight FL environments.

Moving forward, it is essential to develop and refine secure aggregation methods that are both effective against these attacks and computationally efficient. For example, [144] introduces a verifiable aggregation protocol that utilizes a constant-length homomorphic hash

Table 6
Open source projects for the application of lightweight FL.

Paper	Year	About	Link
[113]	2021	Federated Learning with Arduino Nano 33 BLE Sense	https://github.com/MarcMonfort/TinyML-FederatedLearning
[43]	2022	A scheme for privacy-preserving learning on tiny devices.	https://github.com/kavyakvk/TinyFederatedLearning
[34]	2023	Framework for communication-efficient FL with single-step synthetic features compressor.	https://github.com/Soptq/iccv23-3sfc
[51]	2023	Federated divide and cotraining.	https://github.com/vinuni-vishc/fedDCT
[33]	2023	An Efficient and Easy-to-use Federated Learning Framework.	https://github.com/paritybit-ai/XFL
[93]	2024	How to manage tinyML at scale.	https://github.com/Haoyu-R/How-to-Manage-TinyML-at-Scale
[132]	2024	Framework for running federated training and quantization experiments of MLPerfTiny models.	https://github.com/Lawrence-lugs/cidr-ufl

and commitment function, improving user verification while maintaining a lightweight footprint. Additionally, their approach in LightVeriFL [144] employs a one-time aggregate hash recovery mechanism, which not only enhances security but also optimizes the aggregation process for speed, particularly in scenarios where users are intermittently connected.

While lightweight FL offers several advantages in terms of efficiency and privacy, it is crucial to advance research in secure aggregation techniques that do not excessively burden resource-constrained devices. Addressing these challenges will ensure that lightweight FL can be both secure and efficient, enabling its broader adoption in various applications.

5.4. Dynamic environment

Implementing lightweight FL in real-world scenarios and ensuring its longevity presents numerous formidable challenges. Key among these is the critical need for robust resource management protocols and the often overlooked necessity of maintaining up-to-date, high-performing lightweight FL applications [145]. These models quickly become essential parts of practical applications as they are implemented on hardware. To sustain their strong performance, these models must continuously adapt to the latest environmental conditions, which can rapidly render offline-trained models obsolete due to the dynamic nature of deployment environments [37].

A reasonable strategy involves collecting fresh data and regularly updating the device's model to reflect current conditions accurately. However, embedded systems typically grapple with limited memory for storing training data, and the training process itself demands higher peak memory usage, potentially imposing constraints on the size and functionality of deployed models [146]. These complexities underscore the intricate balance required to effectively integrate lightweight FL into practical applications while maintaining optimal performance and adaptability over time.

6. Conclusion

This paper presents a systematic review of lightweight FL techniques and their applications in the IoT. Lightweight FL is particularly well-suited for resource-constrained environments, offering both portability and simplicity. By leveraging the inherent advantages of FL, it enhances privacy protection for IoT participants. As a promising new ML framework, lightweight FL has the potential to redefine IoT architecture. In the research community, significant effort into frameworks for generating compressed models integrated into IoT devices has been put, thereby popularizing lightweight FL applications, but several challenges persist. Issues such as the dynamic nature of production environments pose obstacles to practical deployment. Therefore, ensuring the efficiency of lightweight FL frameworks is crucial for their adaptation across diverse IoT environments.

CRediT authorship contribution statement

Pian Qi: Writing – review & editing, Methodology, Investigation, Data curation. **Diletta Chiaro:** Writing – review & editing, Methodology, Investigation, Formal analysis, Conceptualization. **Francesco Piccialli:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

- G.A.N.D.A.L.F. - Gan Approaches for Non-iiD Aiding Learning in Federations, CUP: E53D23008290006, PNRR - Missione 4 "Istruzione e Ricerca" - Componente C2 Investimento 1.1 "Fondo per il Programma Nazionale di Ricerca e Progetti di Rilevante Interesse Nazionale (PRIN)".

References

[1] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, Y. Gao, A survey on federated learning, Knowl.-Based Syst. 216 (2021) 106775.

[2] P. Qi, D. Chiaro, F. Piccialli, FL-FD: Federated learning-based fall detection with multimodal data fusion, Inf. Fusion 99 (2023) 101890.

[3] X. Zhou, W. Huang, W. Liang, Z. Yan, J. Ma, Y. Pan, I. Kevin, K. Wang, Federated distillation and blockchain empowered secure knowledge sharing for internet of medical things, Inform. Sci. 662 (2024) 120217.

[4] A. Imteaj, M.H. Amini, Leveraging asynchronous federated learning to predict customers financial distress, Intell. Syst. Appl. 14 (2022) 200064.

[5] X. Zhou, Q. Yang, X. Zheng, W. Liang, I. Kevin, K. Wang, J. Ma, Y. Pan, Q. Jin, Personalized federation learning with model-contrastive learning for multimodal user modeling in human-centric metaverse, IEEE J. Sel. Areas Commun. (2024).

[6] D. Agrawal, G. Singh, H. Singh, K. Singh, P. Jha, Y.S. Patel, Fed-avis: A lightweight federated learning assisted anti-vehicle infiltration system, in: Proceedings of the 25th International Conference on Distributed Computing and Networking, 2024, pp. 310–315.

[7] C. Feng, Z. Zhao, Y. Wang, T.Q. Quek, M. Peng, On the design of federated learning in the mobile edge computing systems, IEEE Trans. Commun. 69 (9) (2021) 5902–5916.

[8] C. Xu, Y. Qu, Y. Xiang, L. Gao, Asynchronous federated learning on heterogeneous devices: A survey, Comp. Sci. Rev. 50 (2023) 100595.

[9] A. Imteaj, U. Thakker, S. Wang, J. Li, M.H. Amini, A survey on federated learning for resource-constrained IoT devices, IEEE Internet Things J. 9 (1) (2021) 1–24.

[10] Z. Wei, Q. Pei, N. Zhang, X. Liu, C. Wu, A. Taherkordi, Lightweight federated learning for large-scale IoT devices with privacy guarantee, IEEE Internet Things J. 10 (4) (2021) 3179–3191.

[11] X. Zhou, X. Zheng, X. Cui, J. Shi, W. Liang, Z. Yan, L.T. Yang, S. Shimizu, I. Kevin, K. Wang, Digital twin enhanced federated reinforcement learning with lightweight knowledge distillation in mobile networks, IEEE J. Sel. Areas Commun. (2023).

[12] Z. Zhang, L. Wu, C. Ma, J. Li, J. Wang, Q. Wang, S. Yu, LSFL: A lightweight and secure federated learning scheme for edge computing, IEEE Trans. Inf. Forensics Secur. 18 (2022) 365–379.

[13] J. So, C. He, C.-S. Yang, S. Li, Q. Yu, R. E Ali, B. Guler, S. Avestimehr, Lightsecagg: a lightweight and versatile design for secure aggregation in federated learning, Proc. Mach. Learn. Syst. 4 (2022) 694–720.

[14] P.P. Ray, A review on TinyML: State-of-the-art and prospects, J. King Saud Univ.-Comput. Inf. Sci. 34 (4) (2022) 1595–1623.

[15] L. Colombo, A. Falcetta, M. Roveri, TIFeD: a tiny integer-based federated learning algorithm with direct feedback alignment, in: Proceedings of the Third International Conference on AI-ML Systems, 2023, pp. 1–8.

[16] Google, TensorFlow. URL https://www.tensorflow.org.

[17] Google, TensorFlow lite. URL https://www.tensorflow.org/lite.

[18] J. Lu, Y. Sheng, S. Cao, S. Elnaffar, M.M. Saad, A.M. Seid, A. Erbad, Lyapunov-guided long-term fairness-aware federated learning for collaborative TinyML on edge devices, IEEE Trans. Consum. Electron. (2024).

[19] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, S. Russo, Federated learning for IoT devices: Enhancing TinyML with on-board training, Inf. Fusion 104 (2024) 102189.

[20] H. Cheng, Q. Chen, Y. Liang, G. Zhu, TinyFL: A lightweight federated learning method with efficient memory-and-communication, in: 2023 IEEE Globecom Workshops, GC Wkshps, IEEE, 2023, pp. 608–613.

[21] J. Kim, J. Bang, J. Lee, Adaptive dataset management scheme for lightweight federated learning in mobile edge computing, Sensors 24 (8) (2024) 2579.

[22] N. Llisterri Giménez, M. Monfort Grau, R. Pueyo Centelles, F. Freitag, On-device training of machine learning models on microcontrollers with federated learning, Electronics 11 (4) (2022) 573.

[23] W.Y.B. Lim, N.C. Luong, D.T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, C. Miao, Federated learning in mobile edge networks: A comprehensive survey, IEEE Commun. Surv. Tutor. 22 (3) (2020) 2031–2063.

[24] L. Dutta, S. Bharali, Tinyml meets iot: A comprehensive survey, Internet Things 16 (2021) 100461.

[25] N. Schizas, A. Karras, C. Karras, S. Sioutas, TinyML for ultra-low power AI and large scale IoT deployments: A systematic review, Future Internet 14 (12) (2022) 363.

[26] S.B. Lakshman, N.U. Eisty, Software engineering approaches for tinyml based iot embedded vision: A systematic literature review, in: Proceedings of the 4th International Workshop on Software Engineering Research and Practice for the IoT, 2022, pp. 33–40.

[27] R. Immonen, T. Hämäläinen, Tiny machine learning for resource-constrained microcontrollers, J. Sens. 2022 (1) (2022) 7437023.

[28] K. Pfeiffer, M. Rapp, R. Khalili, J. Henkel, Federated learning for computationally constrained heterogeneous devices: A survey, ACM Comput. Surv. 55 (14s) (2023) 1–27.

[29] V. Rajapakse, I. Karunanayake, N. Ahmed, Intelligence at the extreme edge: A survey on reformable tinyml, ACM Comput. Surv. 55 (13s) (2023) 1–30.

[30] L.G.F. da Silva, D.F. Sadok, P.T. Endo, Resource optimizing federated learning for use with IoT: A systematic review, J. Parallel Distrib. Comput. 175 (2023) 92–108.

[31] N. Khajehali, J. Yan, Y.-W. Chow, M. Fahmideh, A comprehensive overview of IoT-based federated learning: Focusing on client selection methods, Sensors 23 (16) (2023) 7235.

[32] M.J. Page, J.E. McKenzie, P.M. Bossuyt, I. Boutron, T.C. Hoffmann, C.D. Mulrow, L. Shamseer, J.M. Tetzlaff, E.A. Akl, S.E. Brennan, et al., The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, Bmj 372 (2021).

[33] J. Yan, J. Liu, S. Wang, H. Xu, H. Liu, J. Zhou, Heroes: Lightweight federated learning with neural composition and adaptive local update in heterogeneous edge networks, 2023, arXiv preprint arXiv:2312.01617.

[34] Y. Zhou, M. Shi, Y. Li, Y. Sun, Q. Ye, J. Lv, Communication-efficient federated learning with single-step synthetic features compressor for faster convergence, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 5031–5040.

[35] R.-H. Lu, S.-H. Yang, An efficient light-weight federated learning framework implemented on kubernetes and docker, in: 2023 International Conference on Consumer Electronics-Taiwan, ICCE-Taiwan, IEEE, 2023, pp. 685–686.

[36] E. Kurtic, D. Campos, T. Nguyen, E. Frantar, M. Kurtz, B. Fineran, M. Goin, D. Alistarh, The optimal bert surgeon: Scalable and accurate second-order pruning for large language models, 2022, arXiv preprint arXiv:2203.07259.

[37] Z. Duan, Y. Qiao, S. Chen, X. Wang, G. Wu, X. Wang, Lightweight federated reinforcement learning for independent request scheduling in microgrids, in: 2022 IEEE International Conference on Smart Internet of Things, SmartIoT, IEEE, 2022, pp. 208–215.

[38] P. Liu, T. Zhou, Z. Cai, F. Liu, Y. Guo, Leveraging heuristic client selection for enhanced secure federated submodel learning, Inf. Process. Manage. 60 (3) (2023) 103211.

[39] H. Huang, L. Zhang, C. Sun, R. Fang, X. Yuan, D. Wu, Distributed pruning towards tiny neural networks in federated learning, in: 2023 IEEE 43rd International Conference on Distributed Computing Systems, ICDCS, IEEE, 2023, pp. 190–201.

[40] A. Imteaj, M.H. Amini, FedPARL: Client activity and resource-oriented lightweight federated learning model for resource-constrained heterogeneous IoT environment, Front. Commun. Netw. 2 (2021) 657653.

[41] W.X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, 2023, arXiv preprint arXiv:2303.18223.

[42] N.L. Giménez, J. Lee, F. Freitag, H. Vandierendonck, The effects of weight quantization on online federated learning for the IoT: A case study, IEEE Access (2024).

[43] K. Kopparapu, E. Lin, J.G. Breslin, B. Sudharsan, Tinyfedtl: Federated transfer learning on ubiquitous tiny iot devices, in: 2022 IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events, PerCom Workshops, IEEE, 2022, pp. 79–81.

[44] J. Yuan, S. Wang, H. Li, D. Xu, Y. Li, M. Xu, X. Liu, Towards energy-efficient federated learning via INT8-based training on mobile DSPs, in: Proceedings of the ACM on Web Conference 2024, 2024, pp. 2786–2794.

[45] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, 2015, arXiv preprint arXiv:1503.02531.

[46] J. Gou, B. Yu, S.J. Maybank, D. Tao, Knowledge distillation: A survey, Int. J. Comput. Vis. 129 (6) (2021) 1789–1819.

[47] C. Sun, T. Jiang, S. Zonouz, D. Pompili, Fed2kd: Heterogeneous federated learning for pandemic risk assessment via two-way knowledge distillation, in: 2022 17th Wireless On-Demand Network Systems and Services Conference, WONS, IEEE, 2022, pp. 1–8.

[48] Y. Zhu, Y. Wang, Student customized knowledge distillation: Bridging the gap between student and teacher, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 5057–5066.

[49] J. Lin, W.-M. Chen, Y. Lin, C. Gan, S. Han, et al., Mcunet: Tiny deep learning on iot devices, Adv. Neural Inf. Process. Syst. 33 (2020) 11711–11722.

[50] N. Xiong, S. Punnekkat, Tiny federated learning with bayesian classifiers, in: 2023 IEEE 32nd International Symposium on Industrial Electronics, ISIE, IEEE, 2023, pp. 1–6.

[51] Q. Nguyen, H.H. Pham, K.-S. Wong, P. Le Nguyen, T.T. Nguyen, M.N. Do, FedDCT: Federated learning of large convolutional neural networks on resource constrained devices using divide and collaborative training, IEEE Trans. Netw. Serv. Manag. (2023).

[52] M. Das, G. Bagwe, M. Pan, X. Yuan, L. Zhang, Learning, tiny and huge: Heterogeneous model augmentation towards federated tiny learning, in: 2023 International Conference on Machine Learning and Applications, ICMLA, IEEE, 2023, pp. 90–97.

[53] J. Konečnỳ, H.B. McMahan, D. Ramage, P. Richtárik, Federated optimization: Distributed machine learning for on-device intelligence, 2016, arXiv preprint arXiv:1610.02527.

[54] V.C. Gogineni, S. Werner, Y.-F. Huang, A. Kuh, Communication-efficient online federated learning strategies for kernel regression, IEEE Internet Things J. 10 (5) (2022) 4531–4544.

[55] E. Paolini, L. Valcarenghi, N. Andriolli, L. Maggiani, F. Esposito, Enabling lightweight federated learning in nextg wireless networks, in: 2024 IEEE 10th International Conference on Network Softwarization, NetSoft, IEEE, 2024, pp. 304–306.

[56] M. Aledhari, R. Razzak, R.M. Parizi, F. Saeed, Federated learning: A survey on enabling technologies, protocols, and applications, IEEE Access 8 (2020) 140699–140725.

[57] S. Trindade, L.F. Bittencourt, N.L. da Fonseca, Resource management at the network edge for federated learning, Digit. Commun. Netw. (2022).

[58] D. Wu, R. Ullah, P. Harvey, P. Kilpatrick, I. Spence, B. Varghese, Fedadapt: Adaptive offloading for iot devices in federated learning, IEEE Internet Things J. 9 (21) (2022) 20889–20901.

[59] J. Yun, Y. Oh, Y.-S. Jeon, H.V. Poor, Communication-efficient federated learning over capacity-limited wireless networks, 2023, arXiv preprint arXiv:2307.10815.

[60] N. Yoshida, T. Nishio, M. Morikura, K. Yamamoto, R. Yonetani, Hybrid-FL for wireless networks: Cooperative learning mechanism using non-IID data, in: ICC 2020-2020 IEEE International Conference on Communications, ICC, IEEE, 2020, pp. 1–7.

[61] S. Ye, L. Zeng, Q. Wu, K. Luo, Q. Fang, X. Chen, Eco-FL: Adaptive federated learning with efficient edge collaborative pipeline training, in: Proceedings of the 51st International Conference on Parallel Processing, 2022, pp. 1–11.

[62] P. Qi, D. Chiaro, A. Guzzo, M. Ianni, G. Fortino, F. Piccialli, Model aggregation techniques in federated learning: A comprehensive survey, Future Gener. Comput. Syst. (2023).

[63] J. Du, N. Qin, D. Huang, X. Jia, Y. Zhang, Lightweight FL: A low-cost federated learning framework for mechanical fault diagnosis with training optimization and model pruning, IEEE Trans. Instrum. Meas. (2023).

[64] T.P.V. Nguyen, W. Yang, Z. Tang, X. Xia, A.B. Mullens, J.A. Dean, Y. Li, Lightweight federated learning for STIs/HIV prediction, Sci. Rep. 14 (1) (2024) 6560.

[65] Y. Sun, J. Shao, Y. Mao, J. Zhang, Asynchronous semi-decentralized federated edge learning for heterogeneous clients, in: ICC, IEEE, 2022, pp. 5196–5201.

[66] W. Li, X. Cheng, A lightweight federated learning framework in a one-way active communication environment, in: 2022 Euro-Asia Conference on Frontiers of Computer Science and Information Technology, FCSIT, IEEE, 2022, pp. 100–108.

[67] W. Zhu, M. Goudarzi, R. Buyya, FLight: A lightweight federated learning framework in edge and fog computing, Softw. - Pract. Exp. 54 (5) (2024) 813–841.

[68] Y. Cai, W. Xi, Y. Shen, Y. Peng, et al., High-efficient hierarchical federated learning on non-IID data with progressive collaboration, Future Gener. Comput. Syst. 137 (2022) 111–128.

[69] X. Zhou, X. Ye, I. Kevin, K. Wang, W. Liang, N.K.C. Nair, S. Shimizu, Z. Yan, Q. Jin, Hierarchical federated learning with social context clustering-based participant selection for internet of medical things applications, IEEE Trans. Comput. Soc. Syst. 10 (4) (2023) 1742–1751.

[70] A. Gao, J. Zheng, F. Mei, Y. Liu, Substation-level flexible load disaggregation based on hierarchical federated perception algorithm, IEEE Trans. Smart Grid (2024).

[71] A. Alotaibi, A. Barnawi, LightFIDS: Lightweight and hierarchical federated IDS for massive IoT in 6G network, Arab. J. Sci. Eng. 49 (3) (2024) 4383–4399.

[72] R. Firouzi, R. Rahmani, 5G-enabled distributed intelligence based on O-RAN for distributed IoT systems, Sensors 23 (1) (2022) 133.

[73] W. Liu, X. Xu, D. Li, et al., Privacy preservation for federated learning with robust aggregation in edge computing, IEEE Internet Things J. 10 (8) (2023) 7343–7355.

[74] Y. Guo, Y. Wu, Y. Zhu, B. Yang, C. Han, Anomaly detection using distributed log data: A lightweight federated learning approach, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE, 2021, pp. 1–8.

[75] C. Chen, K.I.-K. Wang, P. Li, K. Sakurai, Enhancing security and efficiency: A lightweight federated learning approach, in: International Conference on Advanced Information Networking and Applications, Springer, 2024, pp. 349–359.

[76] M. Aggarwal, V. Khullar, N. Goyal, A. Alammari, M.A. Albahar, A. Singh, Lightweight federated learning for rice leaf disease classification using non independent and identically distributed images, Sustainability 15 (16) (2023) 12149.

[77] N.L. Giménez, J.M. Solé, F. Freitag, Embedded federated learning over a LoRa mesh network, Pervasive Mob. Comput. 93 (2023) 101819.

[78] M.E.E. Alahi, A. Sukkuea, F.W. Tina, A. Nag, W. Kurdthongmee, K. Suwannarat, S.C. Mukhopadhyay, Integration of IoT-enabled technologies and artificial intelligence (AI) for smart city scenario: recent advancements and future trends, Sensors 23 (11) (2023) 5206.

[79] A. Karras, A. Giannaros, C. Karras, L. Theodorakopoulos, C.S. Mammassis, G.A. Krimpas, S. Sioutas, TinyML algorithms for big data management in large-scale IoT systems, Future Internet 16 (2) (2024) 42.

[80] M.S. Ahmad, S.M. Shah, A lightweight mini-batch federated learning approach for attack detection in IoT, Internet Things 25 (2024) 101088.

[81] Z. Wei, Q. Pei, N. Zhang, X. Liu, C. Wu, A. Taherkordi, Lightweight federated learning for large-scale IoT devices with privacy guarantee, IEEE Internet Things J. 10 (4) (2023) 3179–3191, http://dx.doi.org/10.1109/JIOT.2021.3127886.

[82] A. Khalifeh, F. Mazunga, A. Nechibvute, B.M. Nyambo, Microcontroller unit-based wireless sensor network nodes: A review, Sensors 22 (22) (2022) 8937.

[83] K. Char, Internet of Things System Design with Integrated Wireless MCUs, Tech. Rep., Silicon Labs, ARM, 2015.

[84] S.A.R. Zaidi, A.M. Hayajneh, M. Hafeez, Q.Z. Ahmed, Unlocking edge intelligence through tiny machine learning (TinyML), IEEE Access 10 (2022) 100867–100877.

[85] E. Aras, M. Ammar, F. Yang, W. Joosen, D. Hughes, Microvault: Reliable storage unit for iot devices, in: 2020 16th International Conference on Distributed Computing in Sensor Systems, DCOSS, IEEE, 2020, pp. 132–140.

[86] S.J. Johnston, P.J. Basford, C.S. Perkins, H. Herry, F.P. Tso, D. Pezaros, R.D. Mullins, E. Yoneki, S.J. Cox, J. Singer, Commodity single board computer clusters and their applications, Future Gener. Comput. Syst. 89 (2018) 201–212.

[87] E. Gamess, M. Parajuli, S. Shah, Performance evaluation of the KVM hypervisor running on ARM-based single-board computers, Int. J. Comput. Netw. Commun. (IJCNC) 15 (2) (2023) 147–164.

[88] U. Isikdag, U. Isikdag, Internet of Things: Single-board computers, in: Enhanced Building Information Models: Using IoT Services and Integration Patterns, Springer, 2015, pp. 43–53.

[89] A.I. Musa, M. Abdulhamid, B.A. Umar, I.F. Okafor, E. Anne, Novel prototyping development board (source-era) for an encompassing software/hardware production, J. Res. Eng. Comput. Sci. 2 (2) (2024) 23–33.

[90] A. Nayyar, V. Puri, A comprehensive review of beaglebone technology: Smart board powered by ARM, Int. J. Smart Home 10 (4) (2016) 95–108.

[91] S. Aatab, F. Freitag, Integrating a neural network library in embedded federated learning, in: 2023 IEEE 6th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications, CloudTech, IEEE, 2023, pp. 01–07.

[92] H. Ren, D. Anicic, T.A. Runkler, Tinyreptile: Tinyml with federated meta-learning, in: 2023 International Joint Conference on Neural Networks, IJCNN, IEEE, 2023, pp. 1–9.

[93] H. Ren, D. Anicic, X. Li, T. Runkler, On-device online learning and semantic management of TinyML systems, ACM Trans. Embed. Comput. Syst. 23 (4) (2024) 1–32.

[94] C. Dockendorf, S.P. Mohanty, A. Mitra, E. Kougianos, Lite-agro 2.0: Integrating federated and TinyML in pear disease classification IoAT-edge AI, in: 2023 IEEE International Symposium on Smart Electronic Systems, ISES, IEEE, 2023, pp. 429–432.

[95] H. Ren, X. Li, D. Anicic, T.A. Runkler, TinyMetaFed: Efficient federated meta-learning for TinyML, 2023, arXiv preprint arXiv:2307.06822.

[96] C. Gupta, V. Khullar, N. Goyal, K. Saini, R. Baniwal, S. Kumar, R. Rastogi, Cross-silo, privacy-preserving, and lightweight federated multimodal system for the identification of major depressive disorder using audio and electroencephalogram, Diagnostics 14 (1) (2023) 43.

[97] T. Deng, Y. Huang, G. Han, Z. Shi, J. Lin, Q. Dou, Z. Liu, X.-j. Guo, C.P. Chen, C. Han, Feddbl: Communication and data efficient federated deep-broad learning for histopathological tissue classification, IEEE Trans. Cybern. (2024).

[98] N.K. Trivedi, S. Shukla, R.G. Tiwari, A.K. Agarwal, V. Gautam, Liver cancer diagnosis with lightweight federated learning using identically distributed images, in: 2023 12th International Conference on System Modeling & Advancement in Research Trends, SMART, IEEE, 2023, pp. 562–566.

[99] N.K. Trivedi, S. Shukla, A.K. Agarwal, R.G. Tiwari, V. Gautam, Brain tumour diagnosis with lightweight federated learning using identically distributed images, in: 2023 International Conference on Artificial Intelligence for Innovations in Healthcare Industries, ICAIIHI, Vol. 1, IEEE, 2023, pp. 1–5.

[100] N.K. Trivedi, S. Jain, S. Agarwal, Identifying and categorizing Alzheimer's disease with lightweight federated learning using identically distributed images, in: 2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions), ICRITO, IEEE, 2024, pp. 1–5.

[101] D. Chiaro, E. Prezioso, M. Ianni, F. Giampaolo, FL-Enhance: A federated learning framework for balancing non-IID data with augmented and shared compressed samples, Inf. Fusion 98 (2023) 101836.

[102] D. Annunziata, M. Canzaniello, D. Chiaro, S. Izzo, M. Savoia, F. Piccialli, On the dynamics of non-IID data in federated learning and high-performance computing, in: 2024 32nd Euromicro International Conference on Parallel, Distributed and Network-Based Processing, PDP, 2024, pp. 230–237, http://dx.doi.org/10.1109/PDP62718.2024.00039.

[103] W. Lai, Q. Yan, Federated learning for detecting covid-19 in chest ct images: A lightweight federated learning approach, in: 2022 4th International Conference on Frontiers Technology of Information and Computer, ICFTIC, IEEE, 2022, pp. 146–149.

[104] N.K. Trivedi, H. Maheshwari, R.G. Tiwari, A.K. Agarwal, V. Gautam, Lightweight federated learning for COVID-19, pneumonia, and TB from chest X-Ray images, in: 2023 International Conference on Artificial Intelligence for Innovations in Healthcare Industries, ICAIIHI, Vol. 1, IEEE, 2023, pp. 1–6.

[105] N. Nasser, Z.M. Fadlullah, M.M. Fouda, A. Ali, M. Imran, A lightweight federated learning based privacy preserving B5G pandemic response network using unmanned aerial vehicles: A proof-of-concept, Comput. Netw. 205 (2022) 108672.

[106] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, J. Weng, Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search, Comput. Secur. (2024) 103910.

[107] J. Zhang, L. Duan, K. Li, S. Luo, Improved lightweight federated learning network for fault feature extraction of reciprocating machinery, Meas. Sci. Technol. 35 (4) (2024) 045115.

[108] S. Popoola, R. Ande, A. Atayero, M. Hammoudeh, G. Gui, B. Adebisi, Optimized Lightweight Federated Learning for Botnet Detection in Smart Critical Infrastructure, Authorea Preprints, 2023.

[109] X. Zhou, Q. Yang, Q. Liu, W. Liang, K. Wang, Z. Liu, J. Ma, Q. Jin, Spatial–temporal federated transfer learning with multi-sensor data fusion for cooperative positioning, Inf. Fusion 105 (2024) 102182.

[110] T.A. Nguyen, V.K. Kaliappan, S. Jeon, K.-s. Jeon, J.-W. Lee, D. Min, Blockchain empowered federated learning with edge computing for digital twin systems in urban air mobility, in: Asia-Pacific International Symposium on Aerospace Technology, Springer, 2021, pp. 935–950.

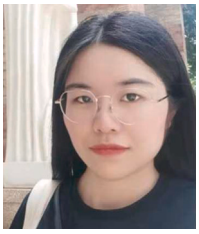
[111] X. Zhou, W. Liang, A. Kawai, K. Fueda, J. She, I. Kevin, K. Wang, Adaptive segmentation enhanced asynchronous federated learning for sustainable intelligent transportation systems, IEEE Trans. Intell. Transp. Syst. (2024).

[112] J. Zeng, K. Zhang, L. Wang, J. Li, FedLVR: a federated learning-based fine-grained vehicle recognition scheme in intelligent traffic system, Multimedia Tools Appl. 82 (24) (2023) 37431–37452.

[113] M.M. Grau, R.P. Centelles, F. Freitag, On-device training of machine learning models on microcontrollers with a look at federated learning, in: Proceedings of the Conference on Information Technology for Social Good, 2021, pp. 198–203.

[114] Z.M. Fadlullah, N. Kato, HCP: Heterogeneous computing platform for federated learning based collaborative content caching towards 6G networks, IEEE Trans. Emerg. Top. Comput. 10 (1) (2020) 112–123.

[115] J. Zheng, J. Xu, H. Du, D. Niyato, J. Kang, J. Nie, Z. Wang, Trust management of tiny federated learning in internet of unmanned aerial vehicles, IEEE Internet Things J. (2024).
[116] L.-l. Fang, H.-r. Hu, W. Pu, J.-q. Bi, Research on uav target recognition technology based on federated learning, in: 2021 2nd International Conference on Computer Engineering and Intelligent Control, ICCEIC, IEEE, 2021, pp. 119–122.
[117] S. Rahim, L. Peng, P.-H. Ho, TinyFDRL-enhanced energy-efficient trajectory design for integrated space-air-ground networks, IEEE Internet Things J. (2024).
[118] K. Raja, K. Kottursamy, V. Ravichandran, S. Balaganesh, K. Dev, L. Nkenyereye, G. Raja, An efficient 6G federated learning-enabled energy-efficient scheme for UAV deployment, IEEE Trans. Veh. Technol. (2024).
[119] N.K. Trivedi, H. Maheshwari, R.G. Tiwari, A.K. Agarwal, V. Gautam, Tomato leaf disease classification with lightweight federated learning using identically distributed images, in: 2023 9th International Conference on Signal Processing and Communication, ICSC, IEEE, 2023, pp. 485–490.
[120] S. Choubey, D. Divya, Lightweight federated transfer learning for plant leaf disease detection and classification across multiclient cross-silo datasets, in: BIO Web of Conferences, Vol. 82, EDP Sciences, 2024, p. 05018.
[121] A. Duttagupta, J. Zhao, S. Shreejith, Exploring lightweight federated learning for distributed load forecasting, in: 2023 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids, SmartGridComm, IEEE, 2023, pp. 1–6.
[122] P. Yuan, R. Huang, J. Zhang, E. Zhang, X. Zhao, Accuracy rate maximization in edge federated learning with delay and energy constraints, IEEE Syst. J. 17 (2) (2022) 2053–2064.
[123] Google, TensorFlow federated. URL https://www.tensorflow.org/federated.
[124] OpenMinded, PySyft. URL https://blog.openmined.org/tag/pysyft/.
[125] TensorOpera, FedML. URL https://www.fedml.ai/.
[126] Flower. URL https://flower.ai/.
[127] WeBank, FATE. URL https://fate.readthedocs.io/en/latest/.
[128] Microsoft, Azure IoT edge. URL https://azure.microsoft.com/it-it/products/iot-edge.
[129] C.N.C. Foundation, KubeEdge. URL https://kubeedge.io/.
[130] Amazon, AWS IoT greengrass. URL https://aws.amazon.com/it/greengrass/.
[131] EdgeX foundry. URL https://www.edgexfoundry.org/.
[132] L.R.A. Quizon, A.B. Alvarez, Federated learning with MLPerfTiny tasks and server-side momentum, in: Proceedings of the 2024 8th International Conference on Machine Learning and Soft Computing, 2024, pp. 32–36.
[133] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, B. He, A survey on federated learning systems: Vision, hype and reality for data privacy and protection, IEEE Trans. Knowl. Data Eng. 35 (4) (2021) 3347–3366.
[134] G. Paolone, D. Iachetti, R. Paesani, F. Pilotti, M. Marinelli, P. Di Felice, A holistic overview of the internet of things ecosystem, IoT 3 (4) (2022) 398–434.
[135] K. Shafique, B.A. Khawaja, F. Sabir, S. Qazi, M. Mustaqim, Internet of things (IoT) for next-generation smart systems: A review of current challenges, future trends and prospects for emerging 5G-IoT scenarios, Ieee Access 8 (2020) 23022–23040.
[136] L. Lao, Z. Li, S. Hou, B. Xiao, S. Guo, Y. Yang, A survey of IoT applications in blockchain systems: Architecture, consensus, and traffic modeling, ACM Comput. Surv. 53 (1) (2020) 1–32.
[137] J.P. Dias, H.S. Ferreira, T.B. Sousa, Testing and deployment patterns for the internet-of-things, in: Proceedings of the 24th European Conference on Pattern Languages of Programs, 2019, pp. 1–8.
[138] J. Hao, J. Chen, P. Chen, Y. Wang, X. Niu, L. Xu, Y. Xia, Efficiently detecting anomalies in IoT: A novel multi-task federated learning method, in: International Conference on Collaborative Computing: Networking, Applications and Worksharing, Springer, 2023, pp. 100–117.
[139] C. Zhang, L. Cui, S. Yu, J. James, A communication-efficient federated learning scheme for iot-based traffic forecasting, IEEE Internet Things J. 9 (14) (2021) 11918–11931.
[140] R. Saha, S. Misra, P.K. Deb, FogFL: Fog-assisted federated learning for resource-constrained IoT devices, IEEE Internet Things J. 8 (10) (2020) 8456–8463.
[141] J. Chin, V. Callaghan, S.B. Allouch, The Internet-of-Things: Reflections on the past, present and future from a user-centered and smart environment perspective, J. Ambient Intell. Smart Environ. 11 (1) (2019) 45–69.
[142] G. Callebaut, G. Leenders, J. Van Mulders, G. Ottoy, L. De Strycker, L. Van der Perre, The art of designing remote iot devices—technologies and strategies for a long battery life, Sensors 21 (3) (2021) 913.
[143] D. Sehrawat, N.S. Gill, Lightweight block ciphers for IoT based applications: a review, Int. J. Appl. Eng. Res. 13 (5) (2018) 2258–2270.
[144] B. Buyukates, J. So, H. Mahdavifar, S. Avestimehr, LightVeriFL: Lightweight and verifiable secure federated learning, in: Workshop on Federated Learning: Recent Advances and New Challenges (in Conjunction with NeurIPS 2022), 2022.
[145] Q.-V. Pham, K. Dev, P.K.R. Maddikunta, T.R. Gadekallu, T. Huynh-The, et al., Fusion of federated learning and industrial Internet of Things: A survey, 2021, arXiv preprint arXiv:2101.00798.
[146] F. Martins Campos de Oliveira, E. Borin, Partitioning convolutional neural networks to maximize the inference rate on constrained IoT devices, Future Internet 11 (10) (2019) 209.



Pian Qi is a Ph.D. student in Mathematics and Computer Science at the University of Naples Federico II. She received the Master Degree in Civil Engineering at the University of Geosciences, Beijing, China in 2020. Her research interests are focused on Deep Learning methodologies, Federated Learning and Data Visualization techniques.

Diletta Chiaro is a Ph.D. student in Mathematics and Computer Science at the University of Naples Federico II. She received the Master Degree in Statistical Sciences at the University of Naples Federico II in 2021. Her research interests are focused on Federated Learning methodologies, Graph Mining and Data Visualization techniques.

Francesco Piccialli is currently Associate Professor of Computer Science and Artificial Intelligence at the Department of Mathematics and Applications “R. Caccioppoli” (DMA), University of Naples Federico II (UNINA), Italy. He received a Laurea Degree (B.Sc.+M.Sc.) in Computer Science and the Ph.D. in Computational and Computer sciences at the University of Naples Federico II. He is the head and co-founder of the M.O.D.A.L. research group that is engaged in cutting-edge on novel methodologies, architectures and applications in the Artificial Intelligence, Machine and Deep Learning fields also focusing on their emerging application domains. He has been involved in research and development projects in the research areas of Artificial Intelligence, Data Science and Internet of Things. He is author of many scientific papers (180+) in international conferences and top-ranking journals (IEEE, ACM, Springer, Elsevier, Wiley, Nature). Currently he serves as Associate Editor of many prestigious, international and well-recognized journals.